Introduction to Machine Learning and Software Analytics to Support Software Engineering

Davide Falessi, PhD

Agenda

- Software Quality
- Software Analytics
- ML to Software Quality
- My Experience in a CMMI Level 5 Context
- Next steps

The importance of Software Quality

- Software bugs globally cost $2.84 trillion dollars.
  - 59.5 billions annually for US



https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2018-report/The-Cost-of-Poor-Quality-Software-in-the-US-2018-Report.pdf

Software evolves extremely fast

Software is written in multiple languages, by many people, over a long period of time in order to fix bugs  , add new features  , and improve code quality  .

Google 50% of the Google's code base changes every month

Microsoft Windows 8 involves 100K+ code changes

And, software is released faster at massive scale

openstack CLOUD SOFTWARE every 6 months

ubuntu every 6 months

Firefox every 6 weeks

facebook every day

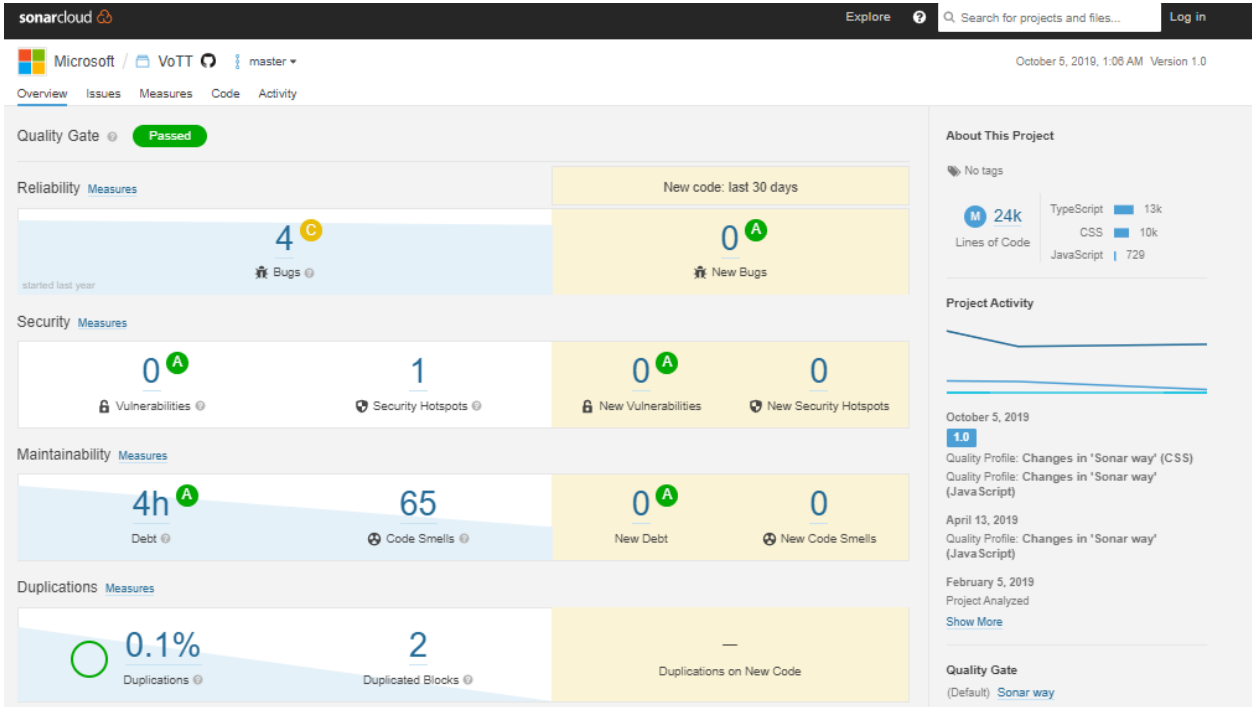


How to prevent bugs?

- Use **static analysis tools** check code quality

sonarqube SpotBugs {PMD Code Quality Matters!}

- Monitor **Technical Debt**
  - Use quality gates



How to find bugs?

Use unit testing to test the functionality correctness,
But manual testing for all files is time-consuming





Use CI/CD to automatically build, test, and merge with confidence



vs



Use code review to find bugs and check code quality



QA IS EXPENSIVE

Google's rules: All changes must be reviewed*

- Large and complex code base: 1 billion lines of code
- > 10K developers in 40+ office locations
- 5K+ projects under active development
- 17K code reviews per day
- 100 million test cases run per day



openstack.

Within 6 months, 1K developers perform 80K+ code reviews (~77 reviews per person) for 30K+ code changes / one release

Given a limited time, how can we effectively prioritise QA resources on the most risky program elements?

https://www.codegrip.tech/productivity/what-is-googles-internal-code-review-process/

https://eclipsecon.org/2013/sites/eclipsecon.org.2013/files/2013-03-24%20Continuous%20Integration%20at%20Google%20Scale.pdf



Software Analytics = Software Data + Data Analytics

GitHub

57 million repositories



26 million developers



100 million pull requests













+ code review + CI logs + test logs + docker config files + others

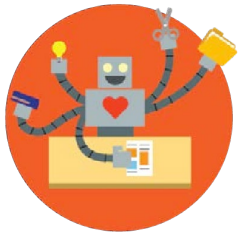


The importance of Software Analytics?



QUALITY IMPROVEMENT

Why do programs crash?
How to prevent bugs in the future?



PRODUCTIVITY IMPROVEMENT

How do continuous integration practices impact team productivity?



PROCESS IMPROVEMENT

How do code review practices and rapid releases impact software quality?



EMPIRICAL THEORY BUILDING

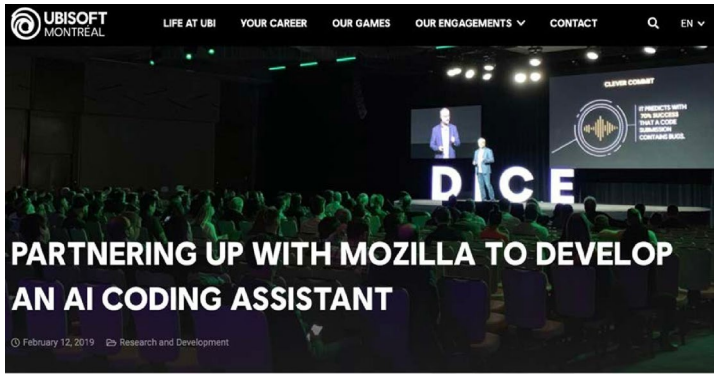
A Theory of Software Quality
A Theory of Effort/Cost Estimation

AI is shaping Software Engineering



IMPROVE SOFTWARE QUALITY

Predict defects, vulnerabilities, malware
Generate test cases

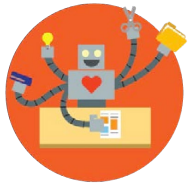




AI is shaping Software Engineering

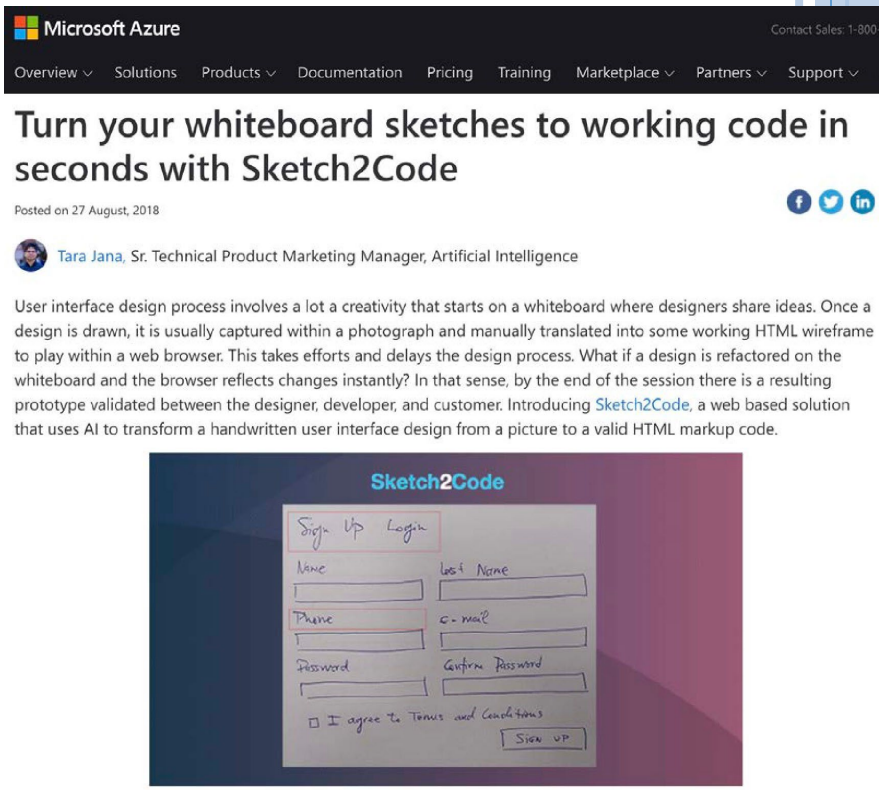
IMPROVE SOFTWARE QUALITY

Predict defects, vulnerabilities, malware
Generate test cases



IMPROVE PRODUCTIVITY

Generating UI/requirements/code/comments
Predict developer/team productivity
Recommend developers/reviewers
Identify developer turnover



ML FOR SOFTWARE DEFECTS

FOCUS ON PREDICTING, EXPLAINING FUTURE SOFTWARE DEFECTS, AND BUILDING EMPIRICAL THEORIES



Predicting future software defects so practitioners can effectively optimize limited resources

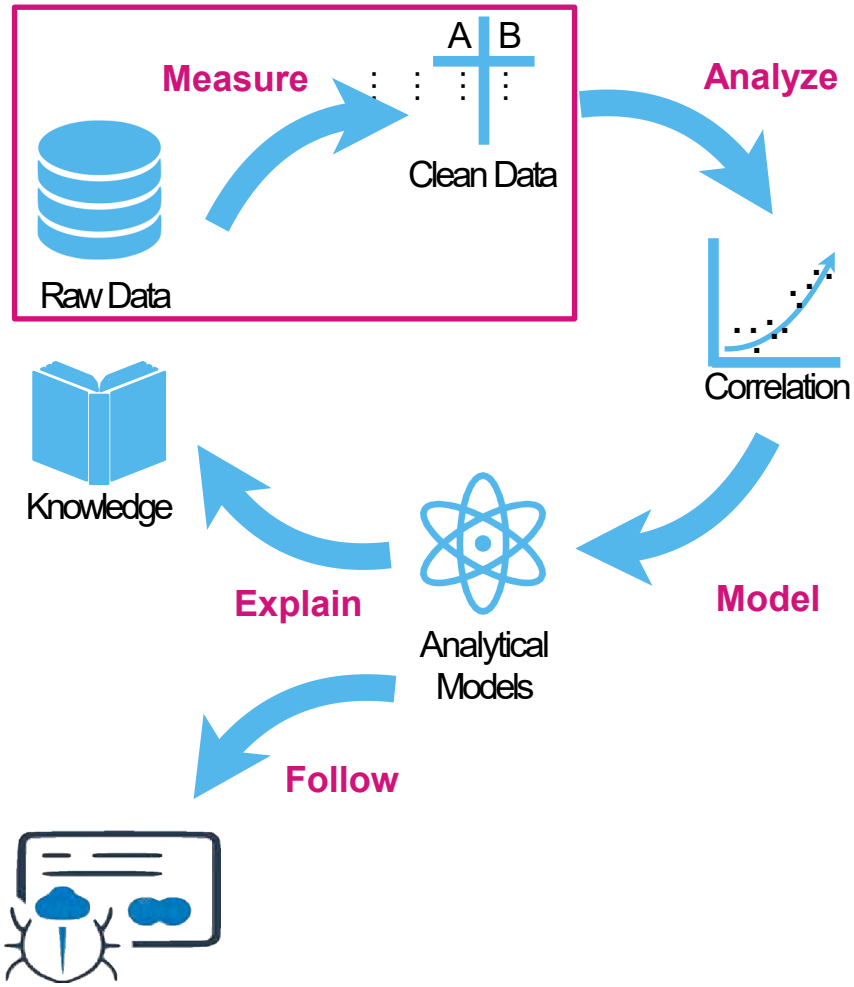


Explaining what makes a software fail so managers can develop the most effective improvement plans

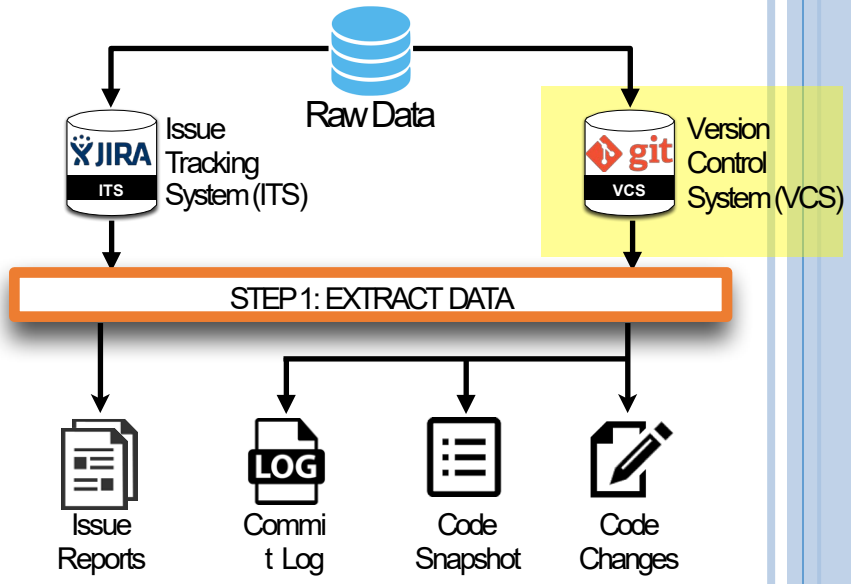


Building empirical-grounded theories of software quality

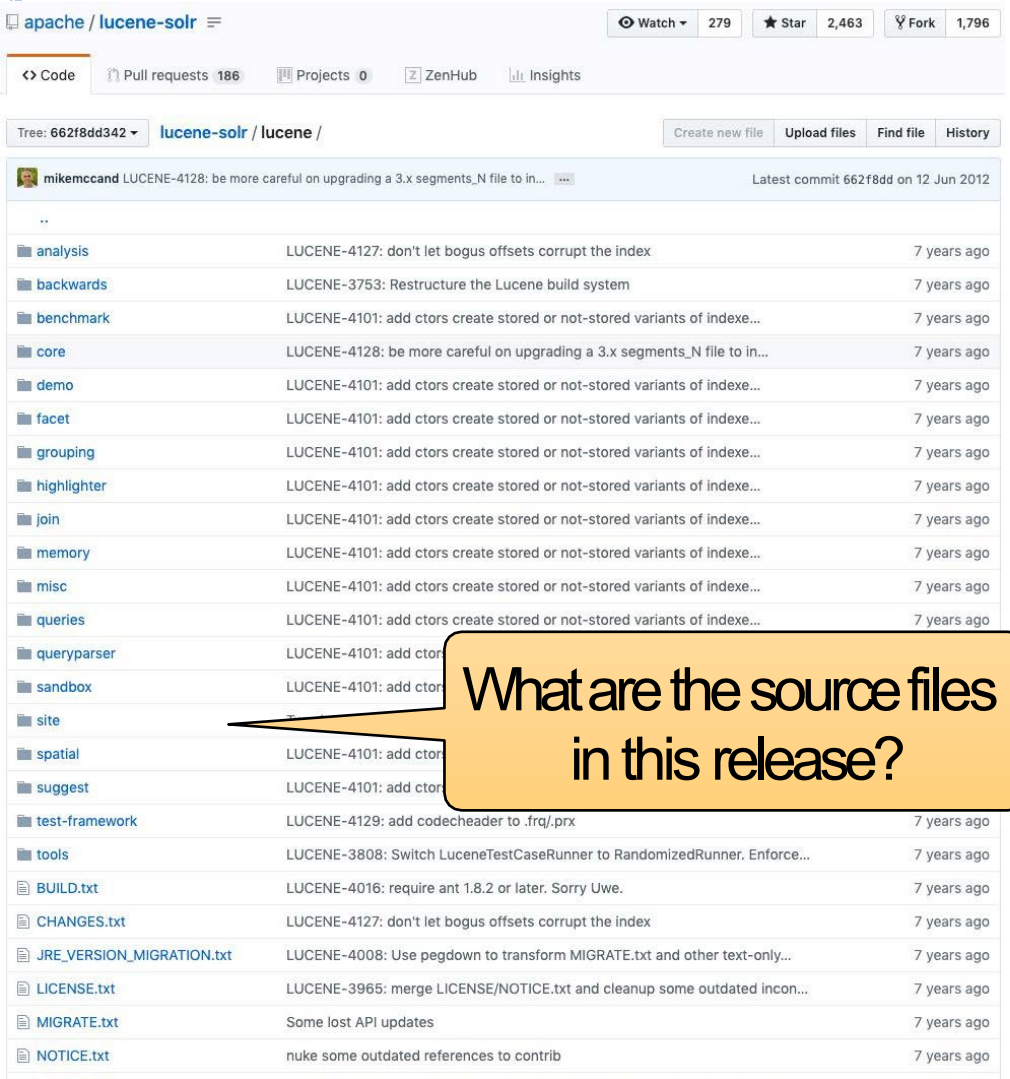
ML FOR SOFTWARE DEFECTS



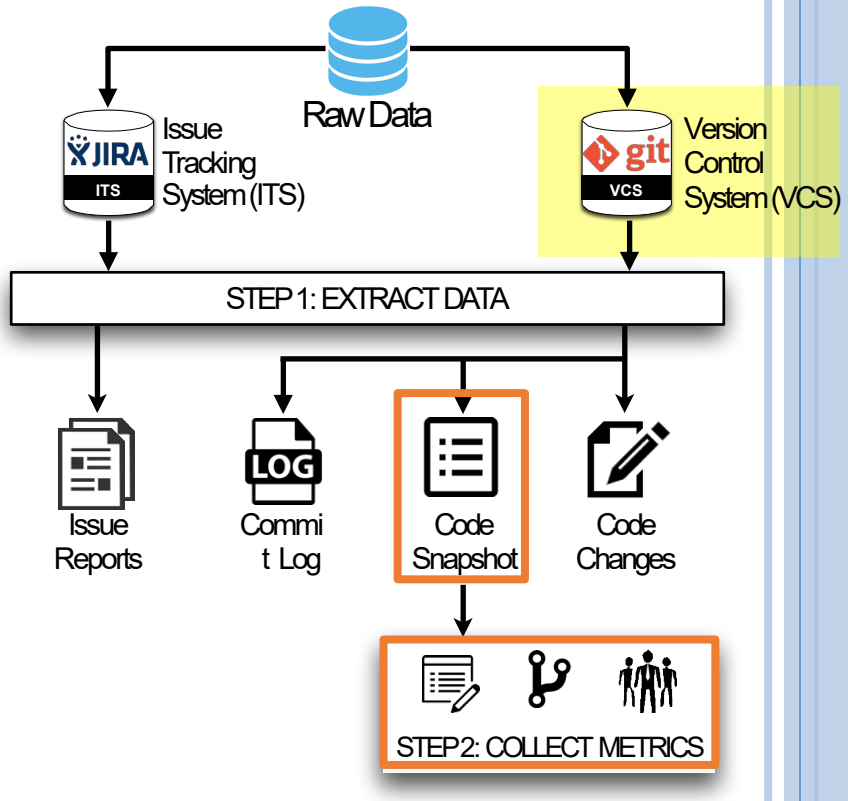
Measure


Raw Data
JIRA
ITS
Issue Tracking System (ITS)
git
VCS
Version Control System (VCS)
STEP 1: EXTRACT DATA
LOG
Issue Reports
Commi t Log
Code Snapshot
Code Changes

MEASURE



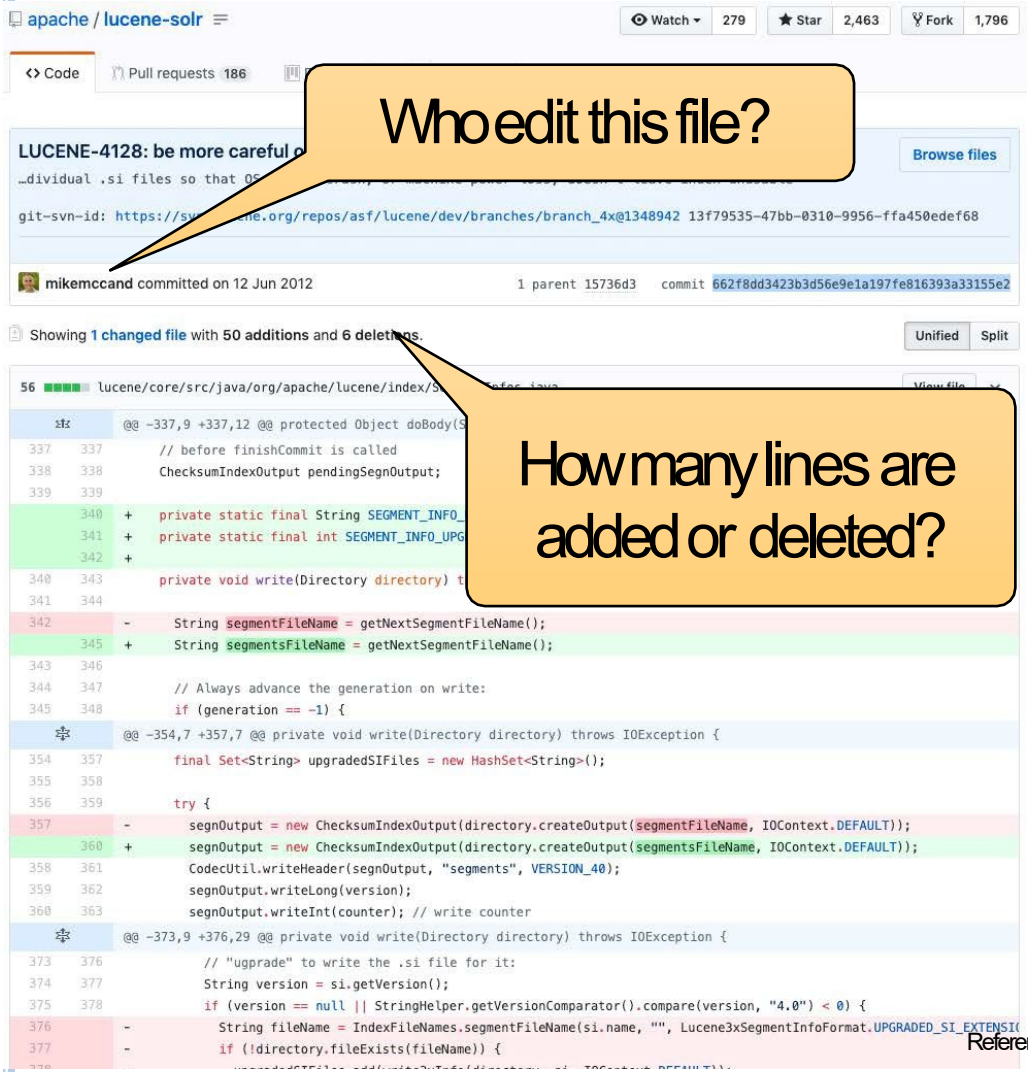
What are the source files in this release?

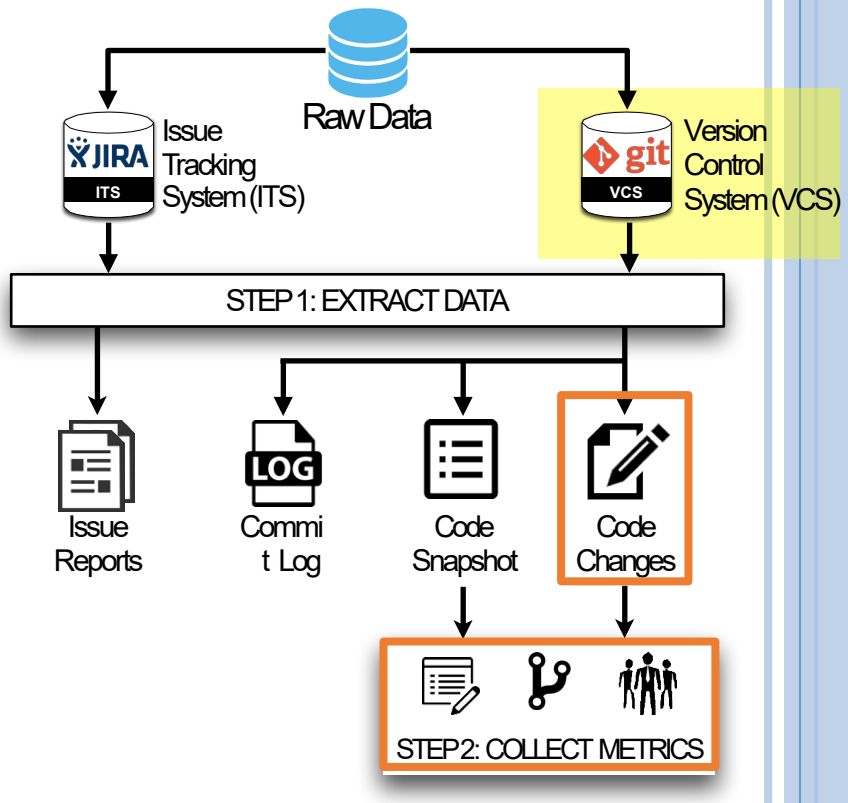


Reference: https://github.com/apache/lucene-solr/tree/662f8dd3423b3d56e9e1a197fe816393a33155e2

https://issues.apache.org/jira/secure/BrowseProjects.jspa?selectedCategory=all&selectedProjectType=all

Measure





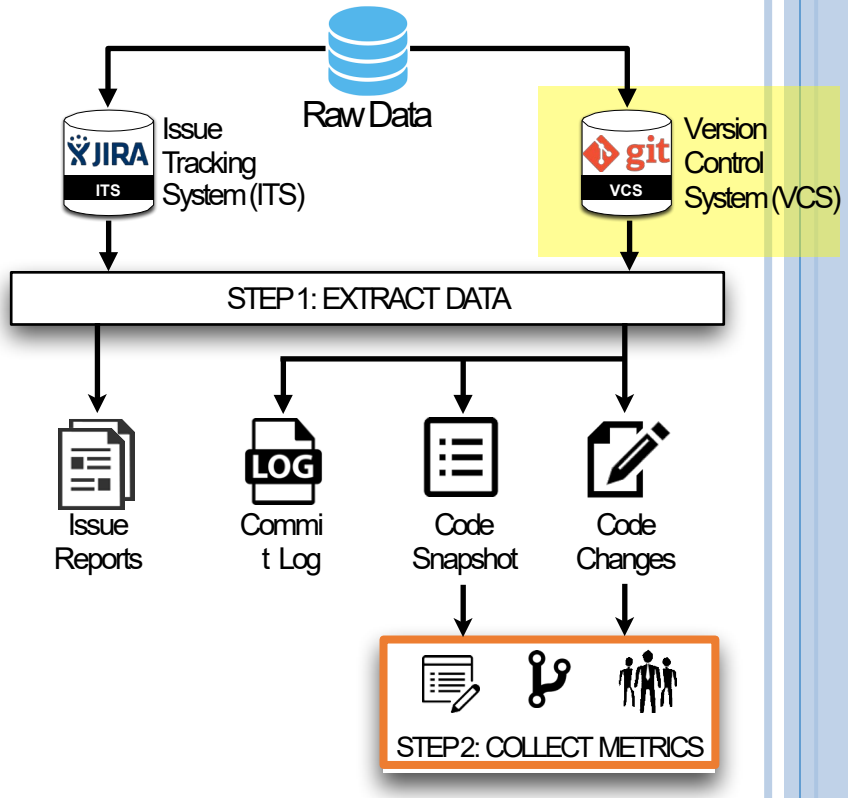
Reference: https://github.com/apache/lucene-solr/commit/662f8dd3423b3d56e9e1a197fe816393a33155e2

Measure

CODE METRICS
Size, Code Complexity, Cognitive Complexity, OO Design (e.g., coupling, cohesion)

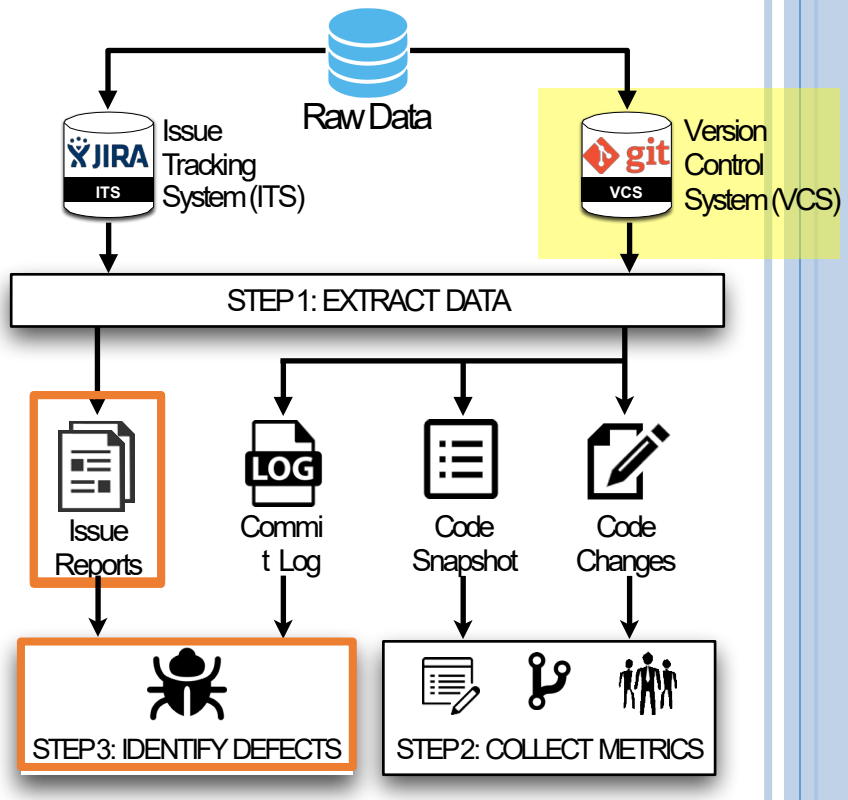
PROCESS METRICS
Development Practices (e.g., #commits, #dev, churn, #pre-release defects, change complexity)

HUMAN FACTORS
Code Ownership, #MajorDevelopers, #MinorDevelopers, Author Ownership, Developer Experience



MEASURE

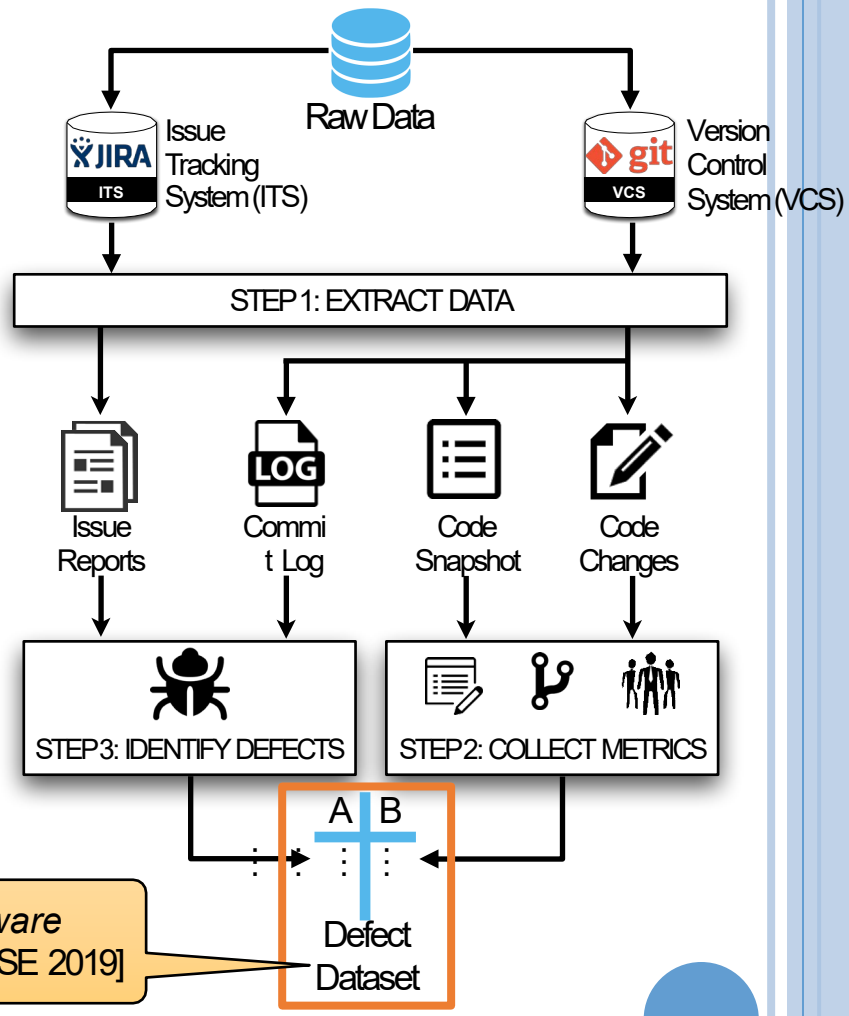




Reference: https://issues.apache.org/jira/browse/LUCENE-4128

Measure



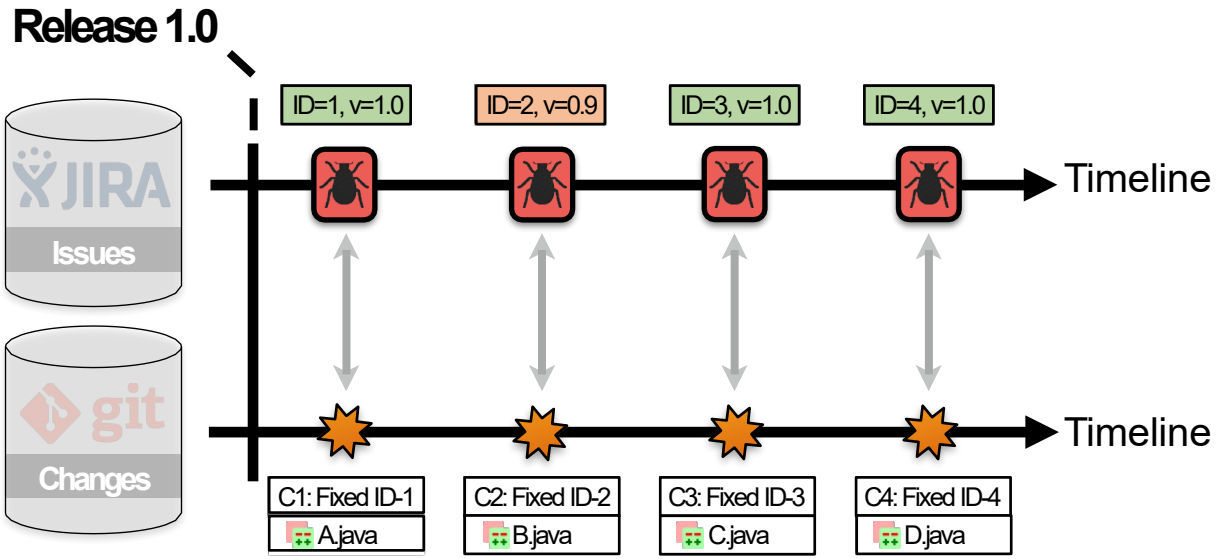


Which files were changed to fix the defect?



Check "*Mining Software Defects*" [Yatish et al., ICSE 2019]

Measure



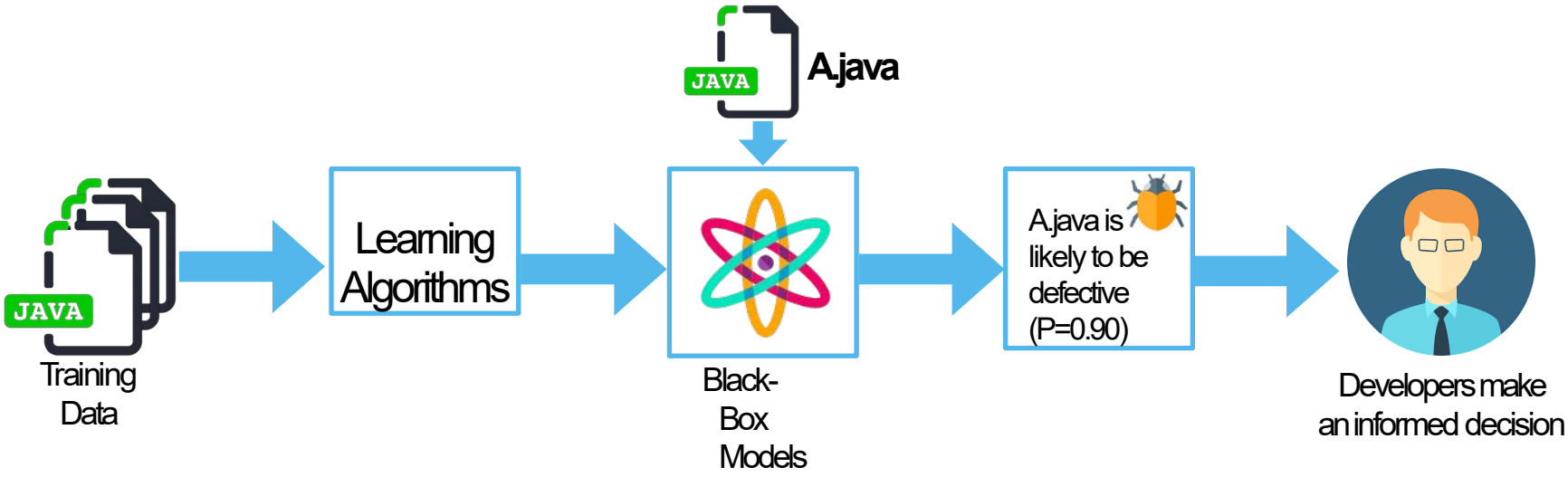
Post-release defects are defined as **modules that are fixed for a defect report that affected a release of interest**

FILE	LABEL
A.java	DEFECTIVE
B.java	**CLEAN**
C.java	DEFECTIVE
D.java	**DEFECTIVE**

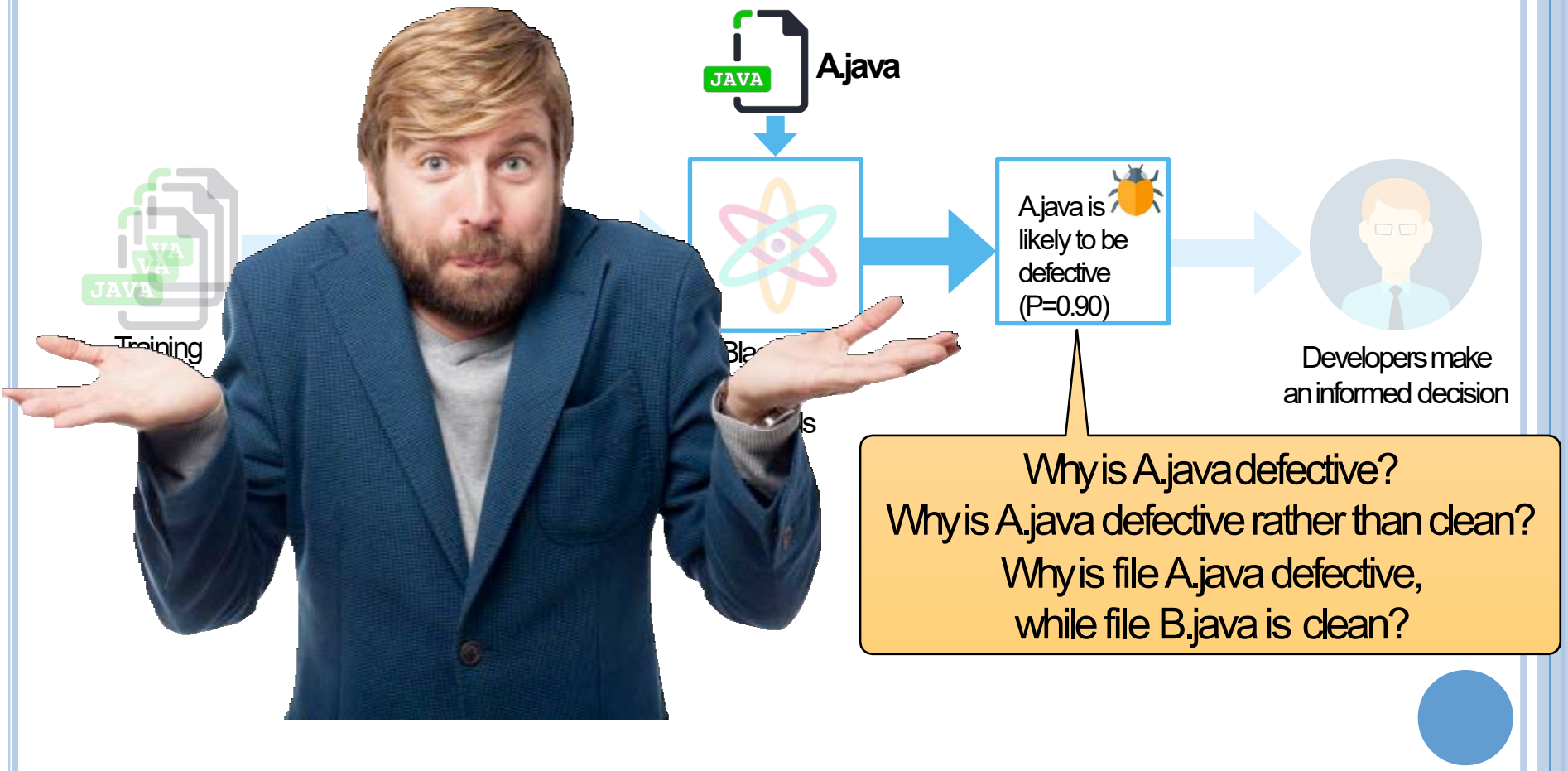
ID indicates a defect report ID, C indicates a commit hash, v indicates affected release(s)

Yatish et al., Mining Software Defects: Should We Consider Affected Releases?, In ICSE'19

MODEL


JAVA
A.java
JAVA
Training Data
Learning Algorithms
Black-Box Models
A.java is likely to be defective (P=0.90)
Developers make an informed decision

Why explaining?





Article 22 of the European Union's General Data Protection Regulation

"The use of data in decision- making that affects an individual or group requires an explanation for any decision made by an algorithm."

http://www.privacy-regulation.eu/en/22.htm

AI/ML BRINGS CONCERNS TO REGULATORS

FAT: FAIRNESS, ACCOUNTABILITY, AND TRANSPARENCY



What if AI-assisted productivity analytics tend to promote males more than females?



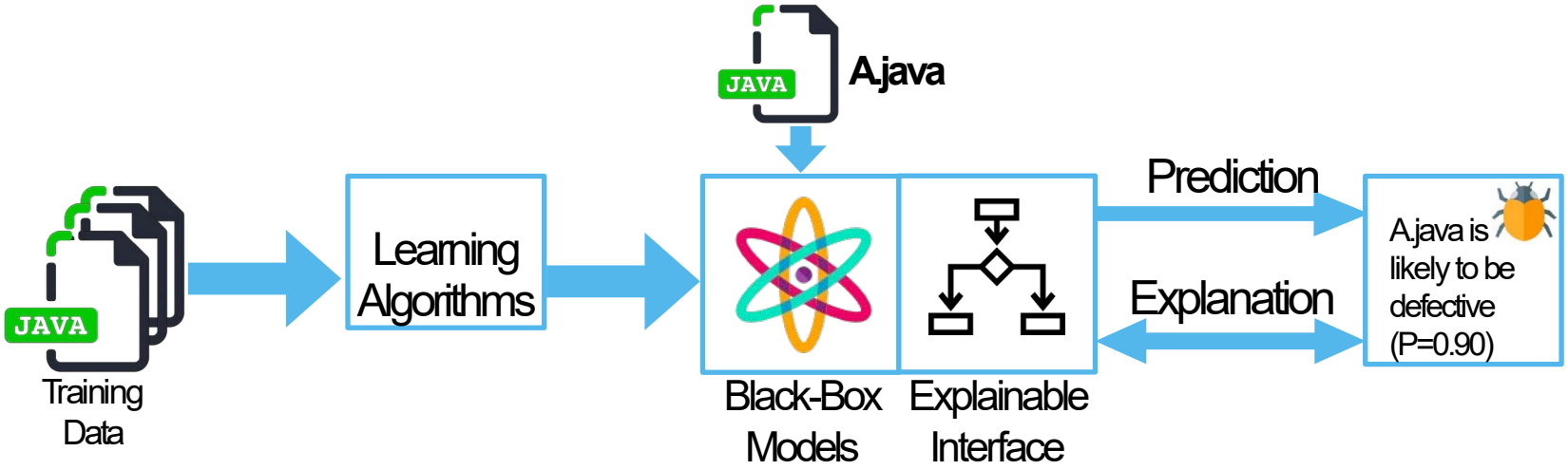
Do the AI systems conform to regulation and legislation?



Do we understand how machines work? Why models make that predictions?

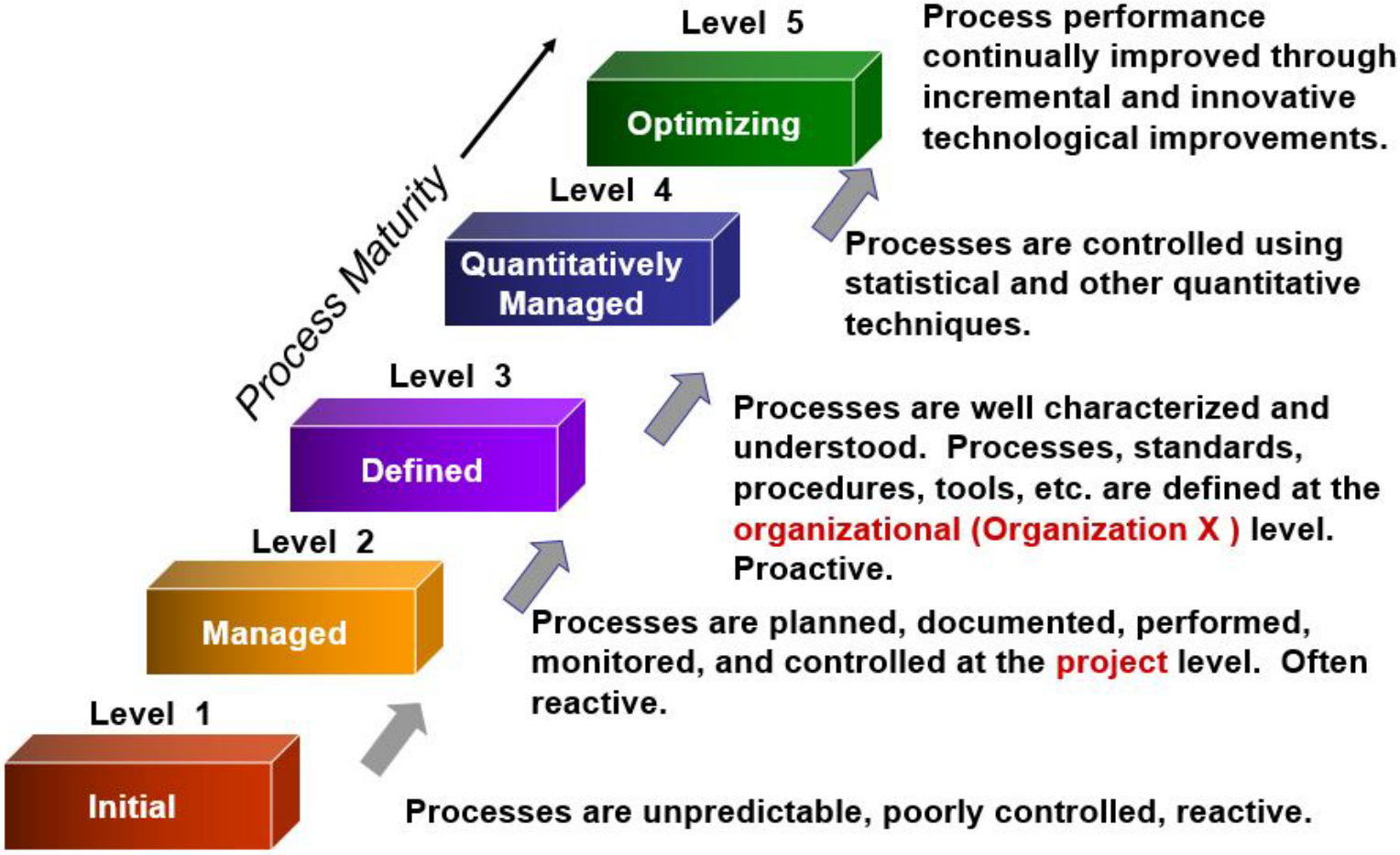
EXPLAIN

A suite of AI/ML techniques that produce accurate predictions, while being able to explain such predictions



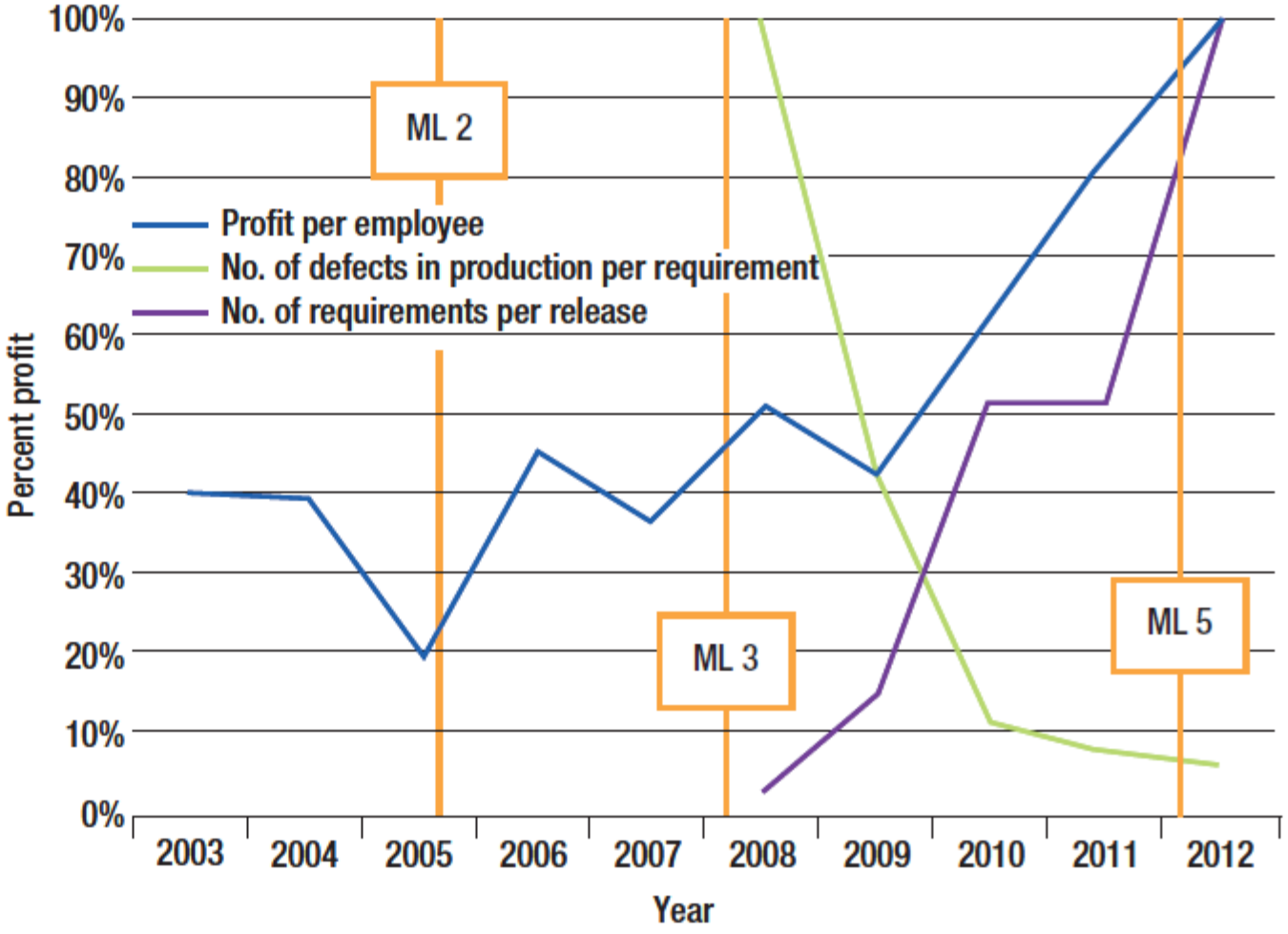
The system provides an explanation that justifies its prediction to the user

My Experience in a CMMI Level 5 Context

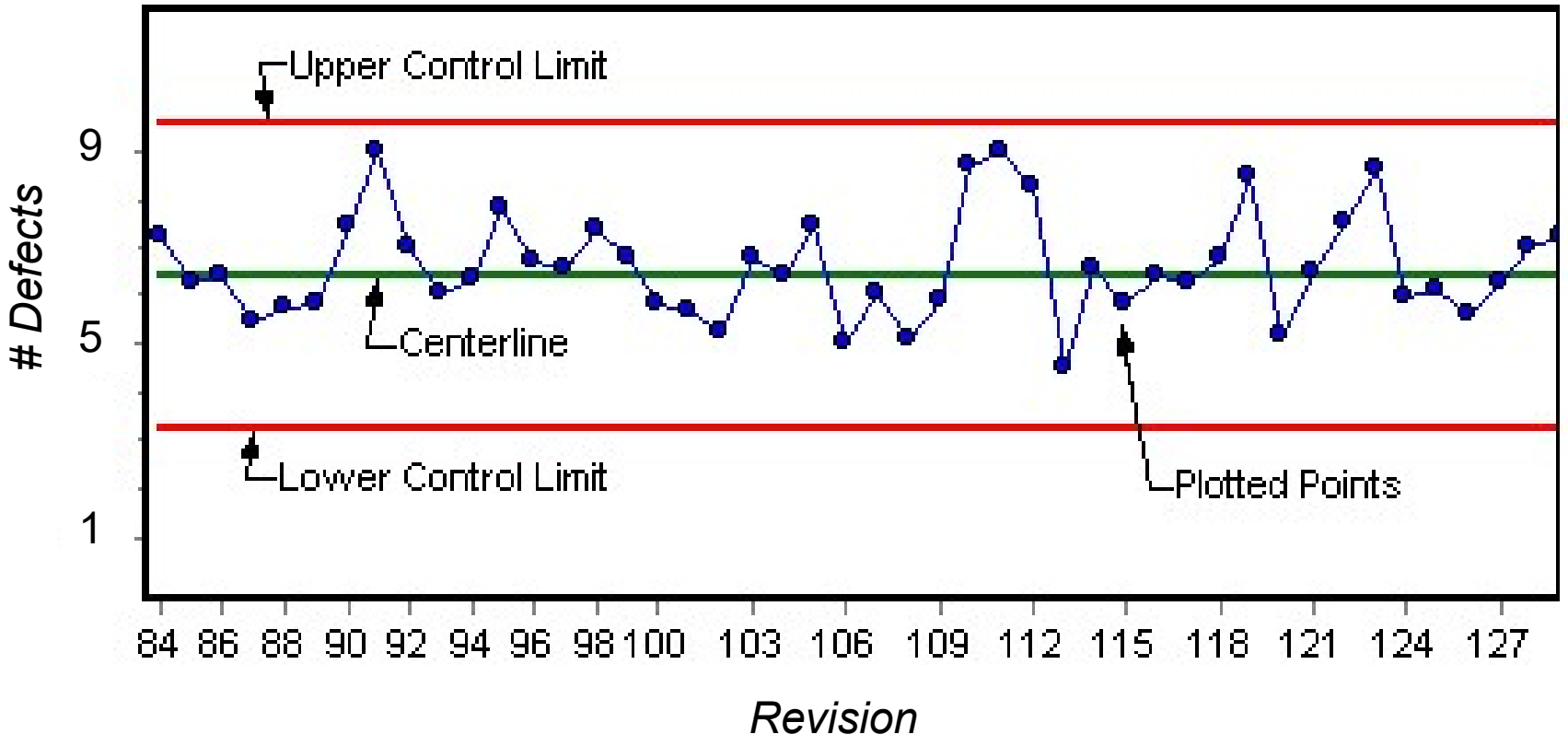


https://www.umsl.edu/~sauterv/analysis/Fall2013Papers/Lanjewar/CMMI.html

My Experience in a CMMI Level 5 Context - Benefits



My Experience in a CMMI Level 5 Context – Software Analytics

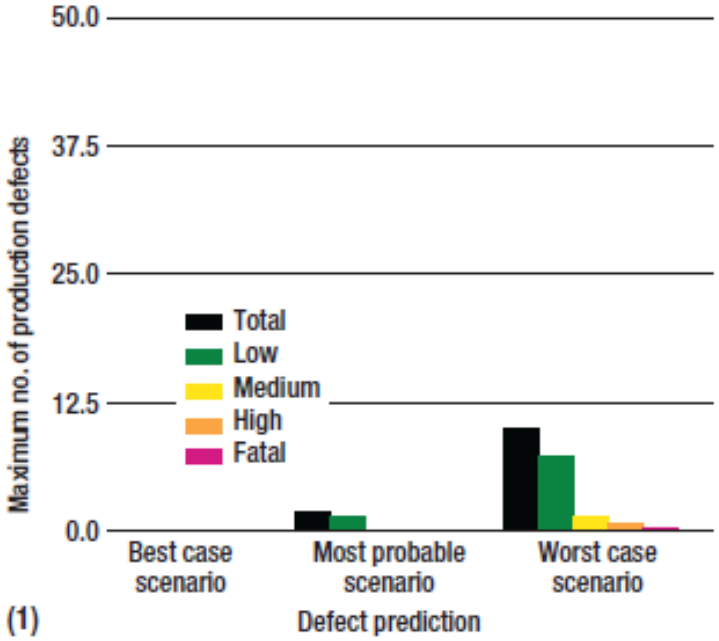


When to use it

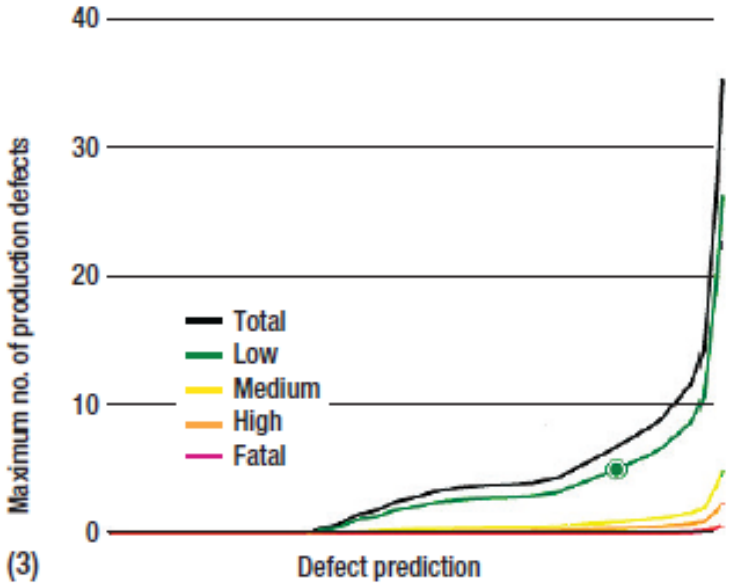
- When **controlling** ongoing processes by finding and correcting problems as they occur.
- When **predicting** the expected range of outcomes from a process.
- When determining whether a process is **stable** (in statistical control).
- When determining whether your quality improvement project should aim to prevent specific problems or to make fundamental changes to the process.

How to do it

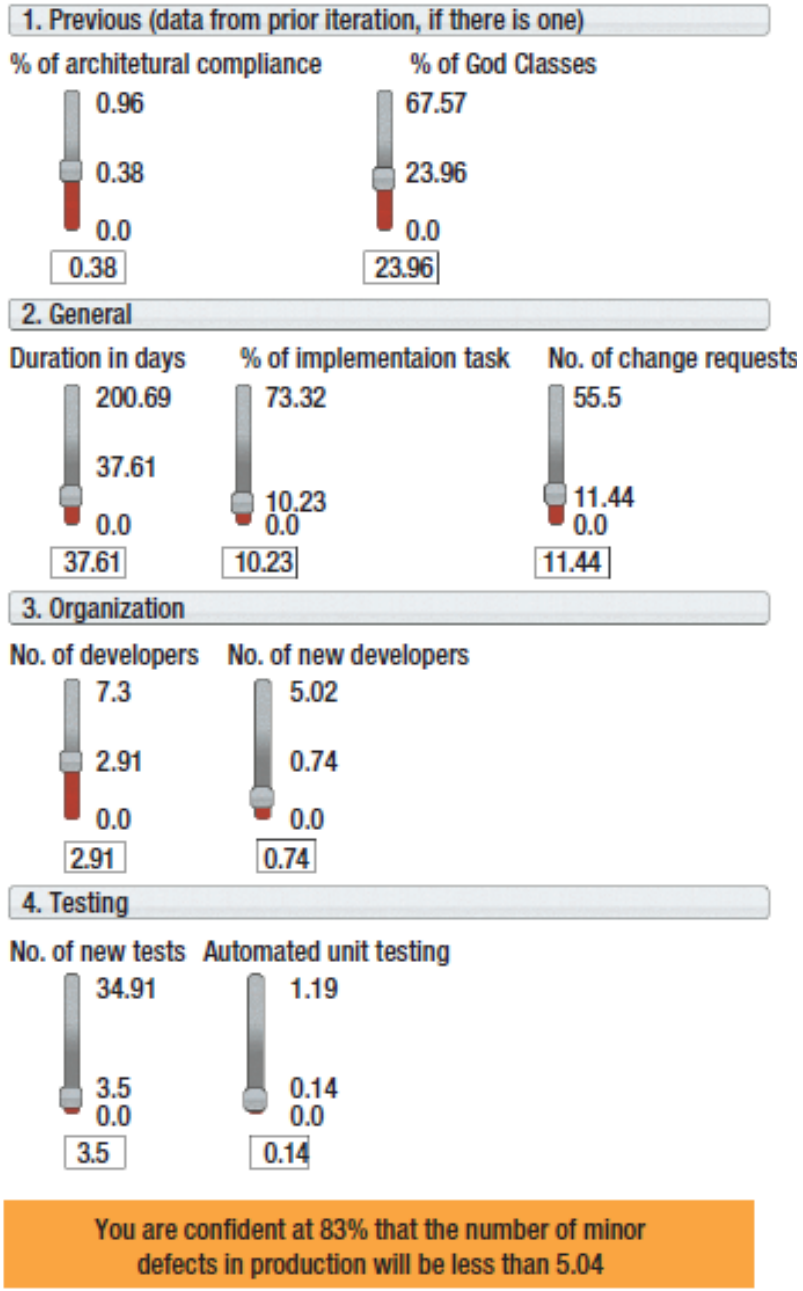
- Select the Y data to visualize. E.g.:
  - Number of commits
  - Number of requirements
  - Number of bugs
- Select the temporal (X) unit. E.g.:
  - Days, Months, weeks
  - Releases
- Collect data from your repository.
- Develop the process chart:
  - Compute mean and standard deviation.
  - Upper limit = mean + 3*STDV
  - Lower limit = mean – 3*STDV
  - Display: points and three lines.


50.0
37.5
25.0
12.5
0.0
Maximum no. of production defects
Total
Low
Medium
High
Fatal
Best case scenario
Most probable scenario
Worst case scenario
Defect prediction

(1)


40
30
20
10
0
Maximum no. of production defects
Total
Low
Medium
High
Fatal
Defect prediction

(3)


1. Previous (data from prior iteration, if there is one)
% of architetural compliance
0.96
0.38
0.0
0.38
% of God Classes
67.57
23.96
0.0
23.96
2. General
Duration in days
200.69
37.61
0.0
37.61
% of implementaion task
73.32
10.23
0.0
10.23
No. of change requests
55.5
11.44
0.0
11.44
3. Organization
No. of developers
7.3
2.91
0.0
2.91
No. of new developers
5.02
0.74
0.0
0.74
4. Testing
No. of new tests
34.91
3.5
0.0
3.5
Automated unit testing
1.19
0.14
0.0
0.14
You are confident at 83% that the number of minor defects in production will be less than 5.04

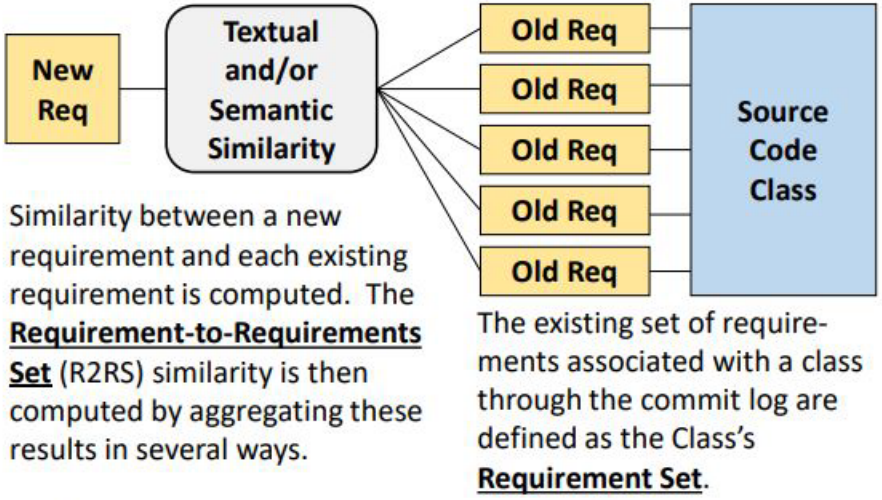
(2)

Predicting Touched Classes

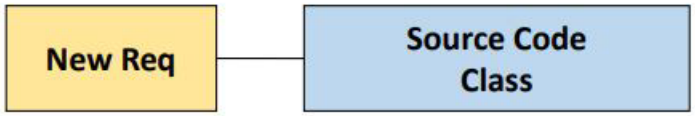
Leveraging Historical Associations between Requirements and Source Code to Identify Impacted Classes

Davide Falessi* *Senior Member, IEEE*, Justin Roll* , Jin L.C. Guo† , and Jane Cleland-Huang‡ *Member, IEEE*
*California Polytechnic State University, San Luis Obispo, CA, 93407
dfalessi@calpoly.edu; jroll@calpoly.edu
†School of Computer Science
McGill University, Montreal, Canada
jguo@cs.mcgill.ca
‡Computer Science and Engineering
University of Notre Dame, South Bend, IN
JaneClelandHuang@nd.edu

Predicting Touched Classes



(a) Requirements to Requirements Set (**R2RS**)



Similarity is directly computed between a new requirement and each existing source code class.

(b) Direct Requirements to Class (**R2C**)

Fig. 1: Textual similarity metrics used by the Class Change Predictor.

Predicting Touched Classes

TABLE 6: Statistics for the four projects.

Project Characteristics	Accumulo	Ignite	Isis	Tika
No. of Releases	8	23	20	27
No. of Commits Linked to Code	7,889	16,571	6,521	2,781
No. of Requirements	145	41	252	49
Average No. of Classes	593	668	2,424	72
No. of Candidate Req–Class Pairs	85,955	27,391	610,833	3,508
% of Positive Instances	3.97%	56.84%	1.94%	7.09%
Earliest Create Date	10/6/11	11/14/14	10/4/10	10/11/07
Latest Commit Date	12/19/14	01/11/16	02/16/16	03/12/14

Predicting Touched Classes

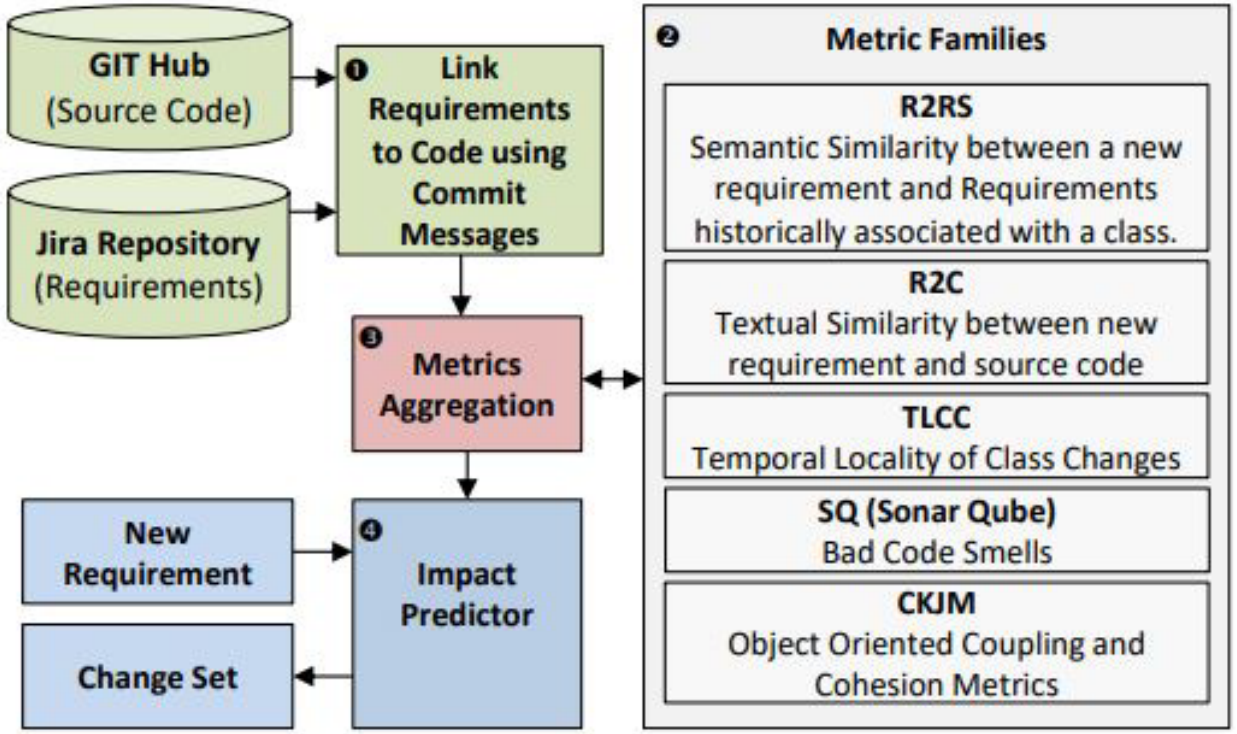


Fig. 12: An overview of the Impact Prediction Process showing the five metric families from which prediction models are trained.

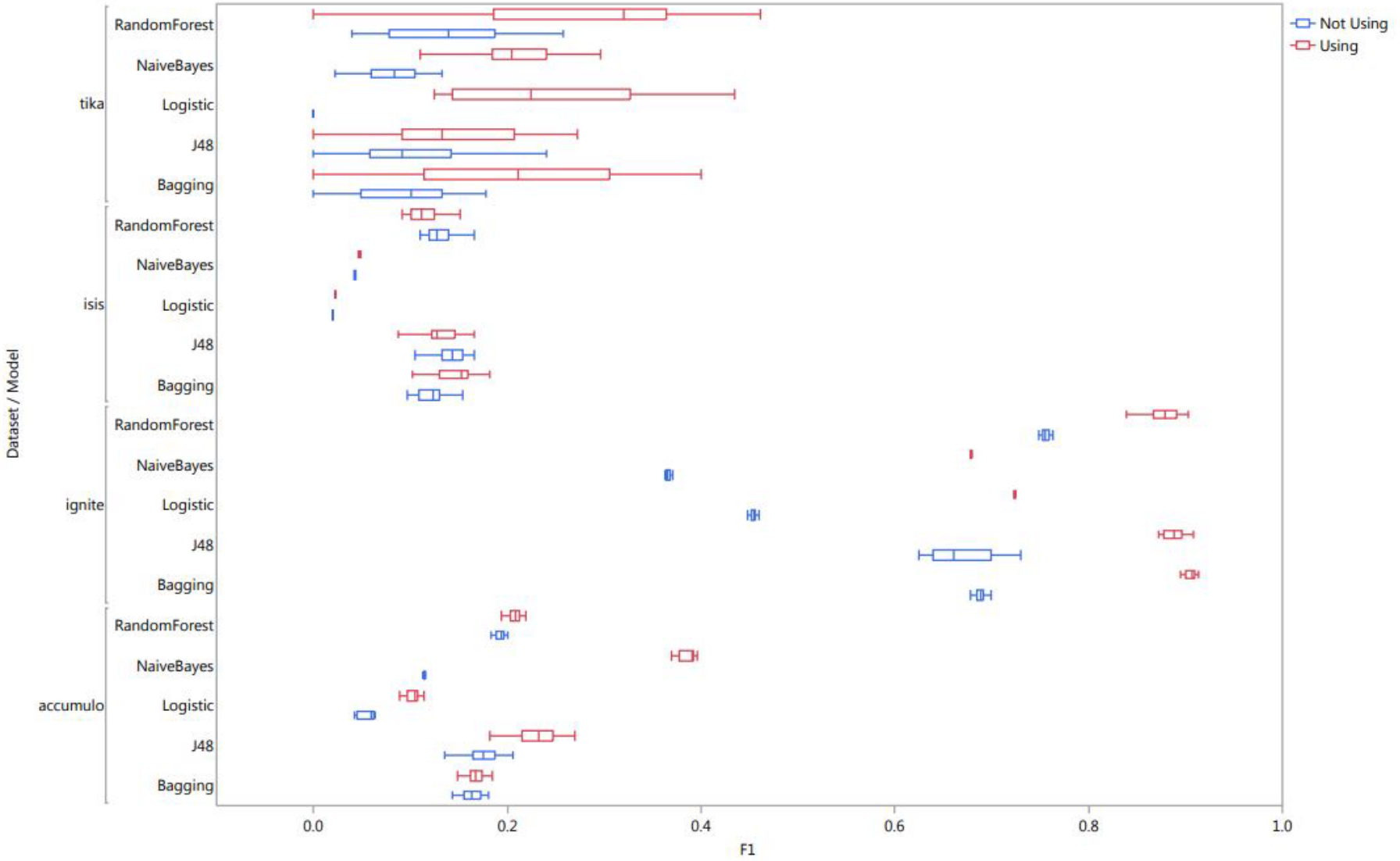


Fig. 11: Distributions of F1, achieved by each of the five classifier, across the 20 samples of each of the four project, by using (blue) or not (red) the R2RS metrics.

Estimating the number of remaining links in traceability recovery

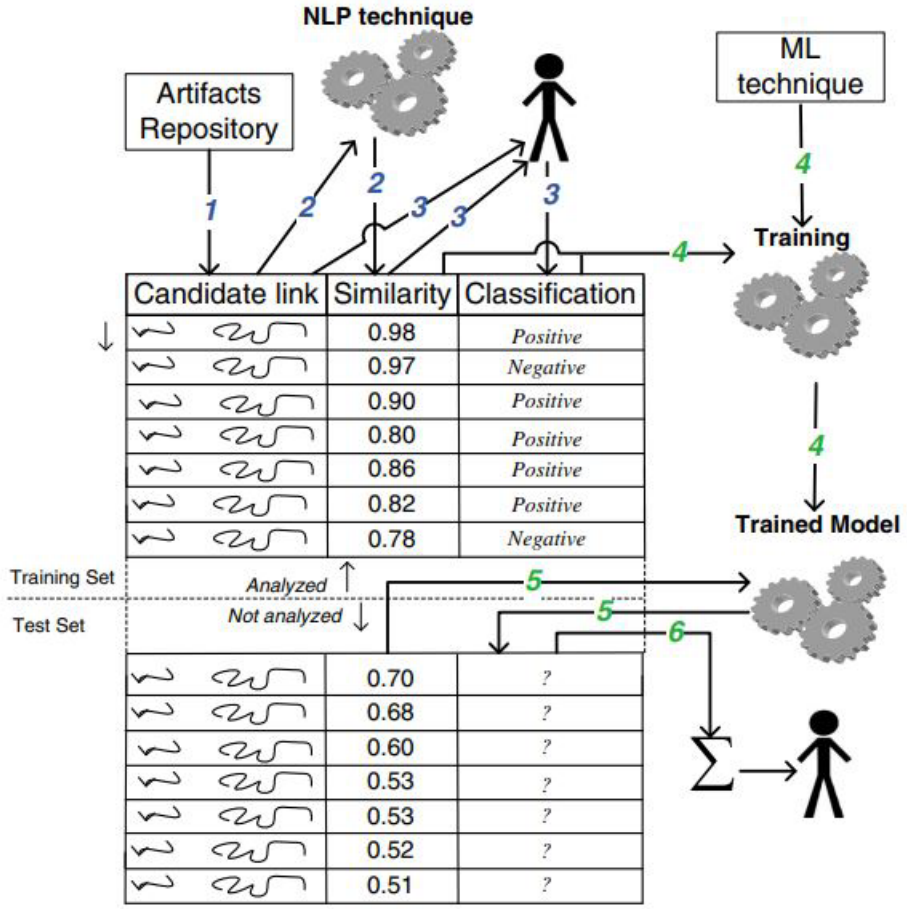


Fig. 2 The ENRL approach: estimating the number of remaining links using NLP techniques and ML classifiers

Estimating the number of remaining links in traceability recovery

Table 1 Characteristics of the three datasets

Project	Artifacts			Candidate Links	Positive Links
	Kind	Number	Total		
Selex SI	Requirements	2,500	2,500	983	138
ETour	Use cases	58	174	6,768	385
	Code classes	116			
EasyClinic	Use cases	30	160	25,440	1,618
	Interaction diagrams	20			
	Source code classes	63			
	Test cases	47			

Estimating the number of remaining links in traceability recovery

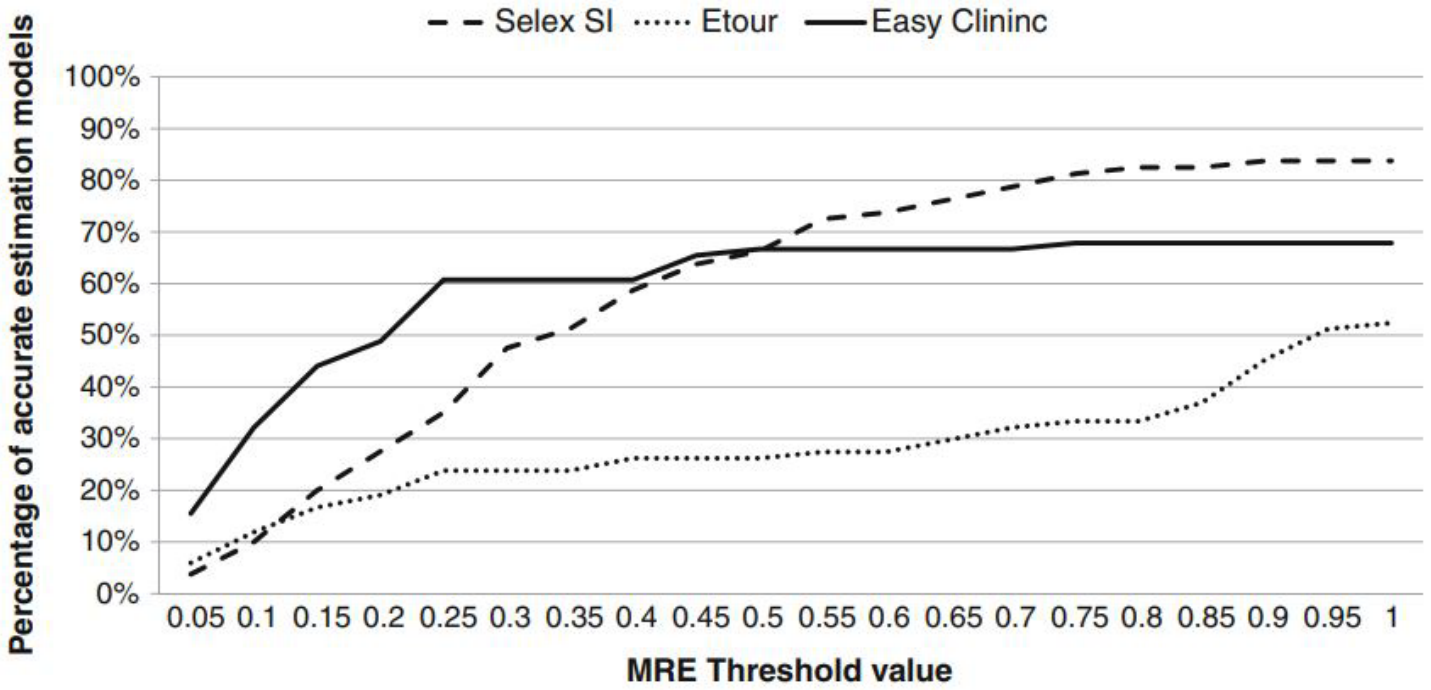


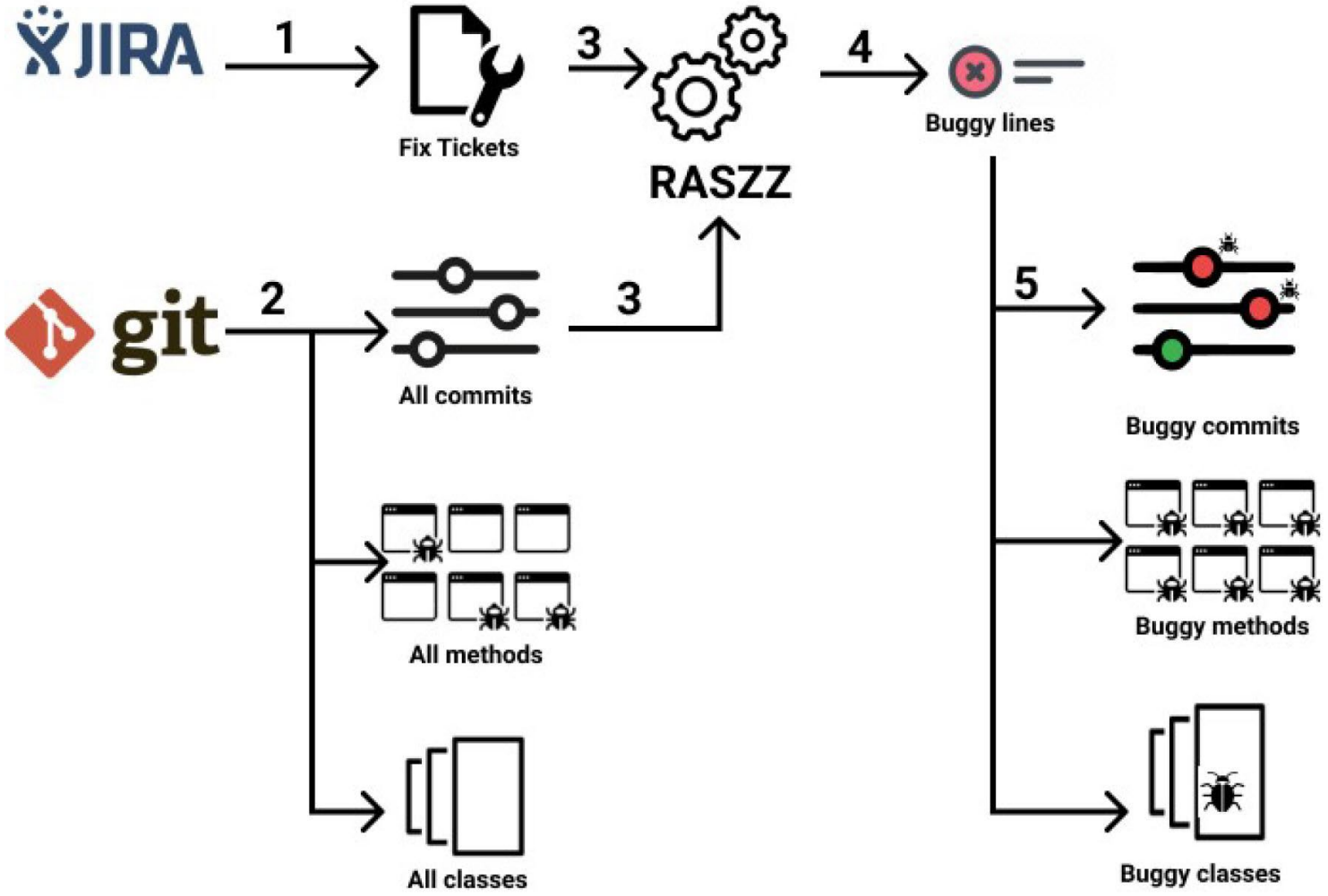
Fig. 3 Relation between MRE threshold and percentage of accurate models obtained

Enhancing the Defectiveness Prediction of Methods and Classes via JIT

Table 1: Details of the datasets used in terms of commits, percent of defective commits, methods, percent of defective methods, classes, percent of defective classes, defects, linkage, and releases.

Dataset	CMT	D. CMT	Meth	D. Meth	CLS	D. CLS	Defects	Linkage	Releases
ARTEMIS	589	34%	83880	0.14%	9425	0.41%	133	82%	1.0.0;1.0.1;1.2.0
DIRSERVER	367	6%	19335	0.11%	3367	1.85%	14	83%	1.01;1.02;1.03;1.04
GROOVY	701	2%	14310	0.12%	1496	1.54%	20	77%	1.0-1;1.0-2;1.0-3;1.0-4;1.0-5
MNG	1835	39%	14519	2.33%	2778	8.8%	582	51%	2.0a1;2.0a2;2.0a3;2.0-1;2.0-2
NUTCH	223	65%	6934	4.86%	1078	13.22%	135	70%	0.7;0.7.1;0.7.2
OPENJPA	538	54%	44997	0.48%	3119	2.65%	263	92%	0.9.0;0.9.6;0.9.7
QPID	1443	44%	43049	3.42%	5423	12.36%	617	82%	M1;M2;M2.1;M3;M4
TIKA	246	29%	1844	0.64%	369	6.32%	69	75%	0.1-incubating;0.2;0.3
ZOOKEEPER	174	21%	6249	0.49%	838	7.01%	27	76%	3.0.0;3.0.1;3.1.0;3.1.1

Enhancing the Defectiveness Prediction of Methods and Classes via JIT



Enhancing the Defectiveness Prediction of Methods and Classes via JIT

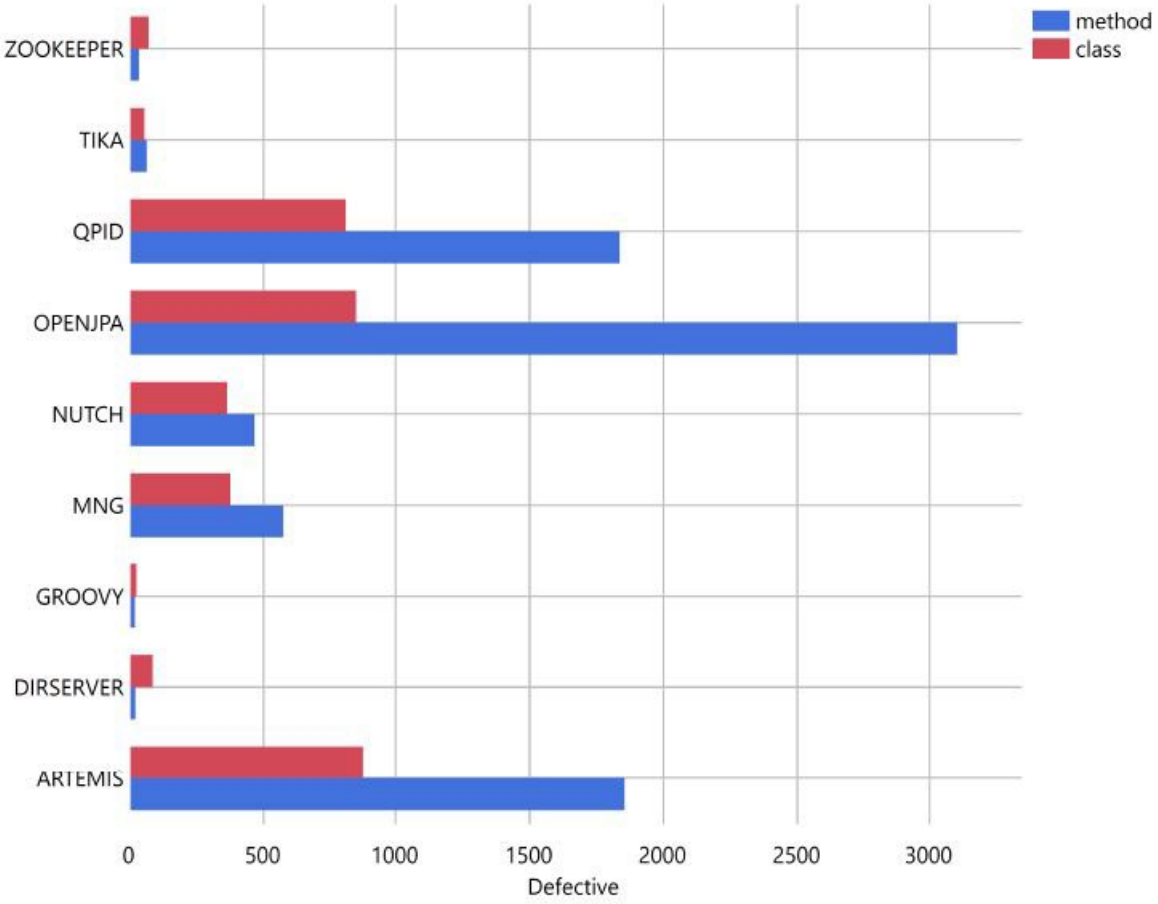


Fig. 4: Number of defective entities (x-axis) in specific projects (y-axis) across different granularities of entities (color).

Enhancing the Defectiveness Prediction of Methods and Classes via JIT

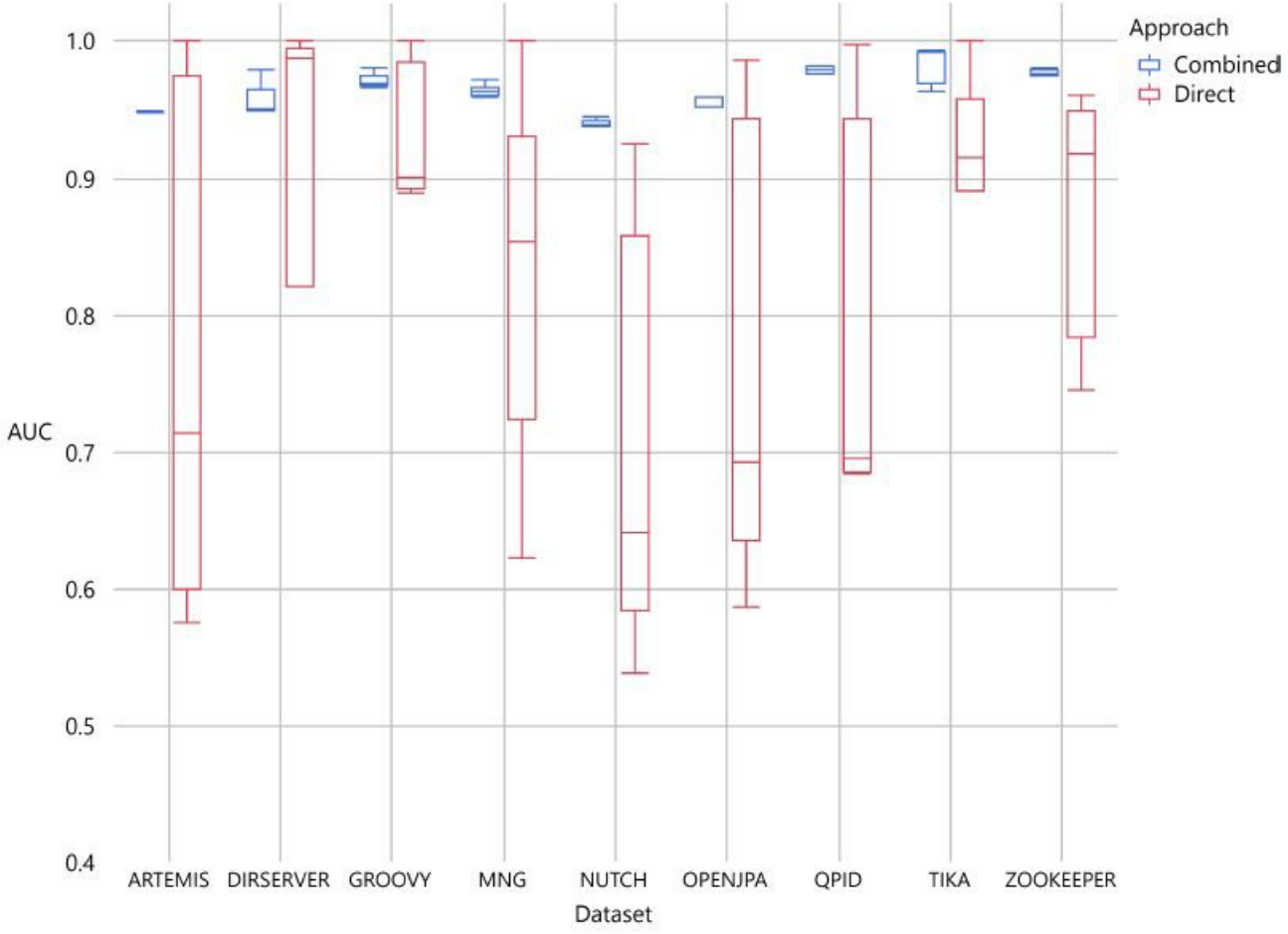


Fig. 6: Distributions across classifiers of AUC values (y-axis) achieved by Combined and Direct (colors) for MDP in our subject projects (x-axis)

Enhancing the Defectiveness Prediction of Methods and Classes via JIT

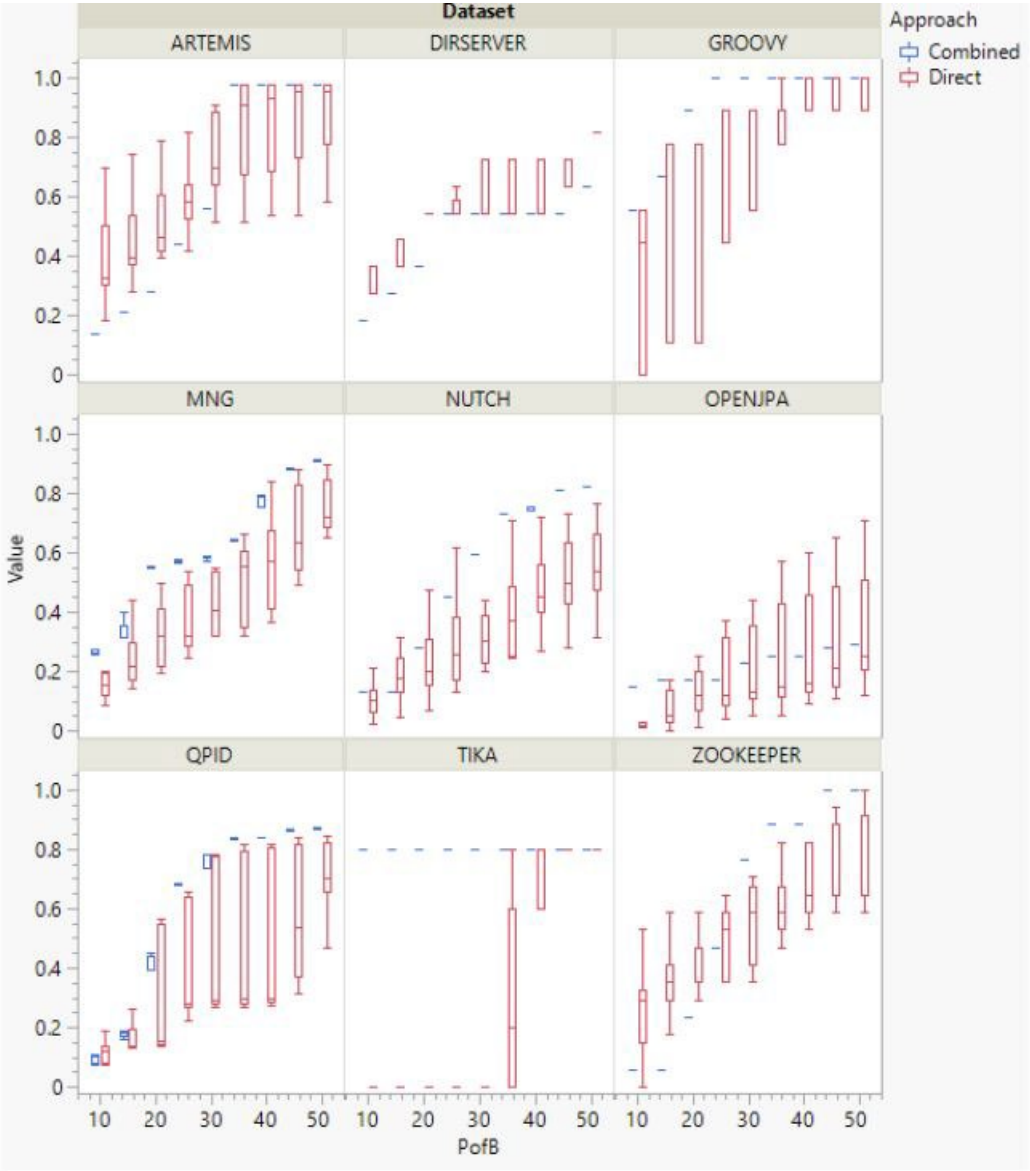


Fig. 7: Distribution across classifiers of PofB values (x-axis) achieved by the Direct and Combined approaches (colors) for MDP in our studied projects (quadrant)

References

- Actionable Analytics for Software Engineering https://www.computer.org/csdl/magazine/so/2018/01/mso2018010051/13rRUxC0Sun
- Achieving and Maintaining CMMI Maturity Level 5 in a Small Organization https://ieeexplore.ieee.org/document/6728932
- Snoring: a noise in defect prediction datasets https://ieeexplore.ieee.org/document/8816788
- Facilitating feasibility analysis: the pilot defects prediction dataset maker https://dl.acm.org/doi/10.1145/3278142.3278147
- Optimizing Prediction Intervals by Tuning Random Forest via Meta-Validation https://arxiv.org/abs/1801.07194

- Thanks to Chakkrit Tantithamthavorn, Monash University, Australia